Database Product Document

Database Overview & Scope

Goal Stats uses PostgreSQL to store teams, Women's World Cup editions, historical matches, editable prediction scenarios, and saved predictions. A Flask REST API supports browsing, scenario creation and updates, unsaved previews, and persistent prediction history. Historical records are imported using validated, repeatable tooling.

Phase 1 implements the five-table database-driven prototype. Player-level data, lineups, automatic live feeds, historical backtesting, and advanced tournament UI remain outside the demonstrated Phase 1 scope; the backend separately provides an unsaved tournament-simulation operation.

Data Sources

Goal Stats uses historical women's international football results compiled by Mart Jürisoo as its primary historical data source. The verified dataset covers 44 national teams across nine FIFA Women's World Cup editions from 1991 to 2023, comprising 348 matches and 1,081 goals. Match scores include extra time but exclude penalty shootout goals. These records support team browsing and historical match comparisons. Predictions are generated separately using hypothetical team ratings and scenario inputs rather than a model trained on the historical dataset.

Database Design

The Goal Stats database is implemented using PostgreSQL and consists of five main tables: Teams, Competitions, Matches, Scenarios, and Predictions. These tables organize historical football data and support the creation, storage, and retrieval of match predictions. Primary and foreign keys establish relationships between records, while database constraints help maintain data accuracy and consistency.

Phase 1 Tables

- Teams: Stores team information and model inputs used for predictions.
- Competitions: Stores competition and tournament information associated with historical matches.
- Matches: Stores real historical match information and results.
- Scenarios: Stores user-created match scenarios and the match state used to generate a prediction.
- Predictions: Stores prediction results generated from a scenario.

Teams

Field	PostgreSQL Data Type	Key	Required	Constraints / Default
team_id	INTEGER	PK	Yes	GENERATED BY DEFAULT AS IDENTITY
name	VARCHAR(120)	—	Yes	Nonblank
name_key	VARCHAR(120)	UQ member	Yes	Nonblank; unique with category
category	VARCHAR(40)	UQ member	Yes	Nonblank; unique with name_key
offense	DOUBLE PRECISION	—	No	0–100 when supplied; ratings attribution
defense	DOUBLE PRECISION	—	No	0–100 when supplied; ratings attribution
form	DOUBLE PRECISION	—	No	0–100 when supplied; ratings attribution
star_power	DOUBLE PRECISION	—	No	0–100 when supplied; ratings attribution
clutch	DOUBLE PRECISION	—	No	0–100 when supplied; ratings attribution
fifa_rank	INTEGER	—	No	Greater than 0 when supplied; ranking attribution
ratings_source	TEXT	—	No	Must be non-NULL and nonblank if any rating is supplied
ranking_source	TEXT	—	No	Must be non-NULL and nonblank if fifa_rank or ranking_date is supplied
ranking_date	DATE	—	No	Ranking attribution

Teams constraint: UNIQUE(name_key, category). Optional ratings require a nonblank ratings_source; optional rank/date values require a nonblank ranking_source.

Competitions

Field	PostgreSQL Data Type	Key	Required	Constraints / Default
competition_id	INTEGER	PK	Yes	GENERATED BY DEFAULT AS IDENTITY
name	VARCHAR(160)	—	Yes	Nonblank
name_key	VARCHAR(160)	UQ member	Yes	Nonblank; composite uniqueness below
edition_label	VARCHAR(40)	UQ member	Yes	Nonblank; composite uniqueness below
category	VARCHAR(40)	UQ member	Yes	Nonblank; composite uniqueness below

Matches

Field	PostgreSQL Data Type	Key	Required	Constraints / Default
match_id	INTEGER	PK	Yes	GENERATED BY DEFAULT AS IDENTITY
competition_id	INTEGER	FK	Yes	References competitions.competition_id; ON DELETE RESTRICT
home_team_id	INTEGER	FK	Yes	References teams.team_id; ON DELETE RESTRICT; differs from away_team_id
away_team_id	INTEGER	FK	Yes	References teams.team_id; ON DELETE RESTRICT; differs from home_team_id
match_date	DATE	—	Yes	No additional CHECK
home_score	SMALLINT	—	Yes	Greater than or equal to 0
away_score	SMALLINT	—	Yes	Greater than or equal to 0
score_basis	VARCHAR(64)	—	Yes	Nonblank
neutral	BOOLEAN	—	No	NULL allowed; no default
city	TEXT	—	No	NULL allowed
country	TEXT	—	No	NULL allowed
source	VARCHAR(100)	UQ member	Yes	Nonblank; unique with source_record_key
source_record_key	VARCHAR(64)	UQ member	Yes	Exactly 64 lowercase hexadecimal characters; unique with source

source_version	TEXT	—	Yes	Nonblank
source_file_sha256	VARCHAR(64)	—	Yes	Exactly 64 lowercase hexadecimal characters
source_row	INTEGER	—	Yes	Greater than 0
source_url	TEXT	—	Yes	Nonblank
source_home_team	TEXT	—	Yes	Nonblank
source_away_team	TEXT	—	Yes	Nonblank
source_tournament	TEXT	—	Yes	Nonblank
imported_at	TIMESTAMP WITH TIME ZONE	—	Yes	DEFAULT now()

Scenarios

Field	PostgreSQL Data Type	Key	Required	Constraints / Default
scenario_id	INTEGER	PK	Yes	GENERATED BY DEFAULT AS IDENTITY
title	VARCHAR(120)	—	No	NULL allowed; no nonblank CHECK
team1_id	INTEGER	FK	Yes	References teams.team_id; ON DELETE RESTRICT; differs from team2_id
team2_id	INTEGER	FK	Yes	References teams.team_id; ON DELETE RESTRICT; differs from team1_id
match_id	INTEGER	FK	No	References matches.match_id; ON DELETE RESTRICT
created_at	TIMESTAMP WITH TIME ZONE	—	Yes	DEFAULT now()
match_minute	INTEGER	—	Yes	0–120
team1_goals	INTEGER	—	Yes	0–1000
team1_shots	INTEGER	—	Yes	0–1000
team1_shots_on_target	INTEGER	—	Yes	0–1000
team1_big_chances	INTEGER	—	Yes	0–1000
team1_defender_blocks	INTEGER	—	Yes	0–1000
team1_goalkeeper_saves	INTEGER	—	Yes	0–1000
team1_corners	INTEGER	—	Yes	0–1000
team1_yellow_cards	INTEGER	—	Yes	0–1000
team1_red_cards	INTEGER	—	Yes	0–11
team1_possession	DOUBLE PRECISION	—	Yes	0–100; combined possession rule
team1_modifier_total	DOUBLE PRECISION	—	Yes	−3000–3000; DEFAULT 0
team2_goals	INTEGER	—	Yes	0–1000
team2_shots	INTEGER	—	Yes	0–1000

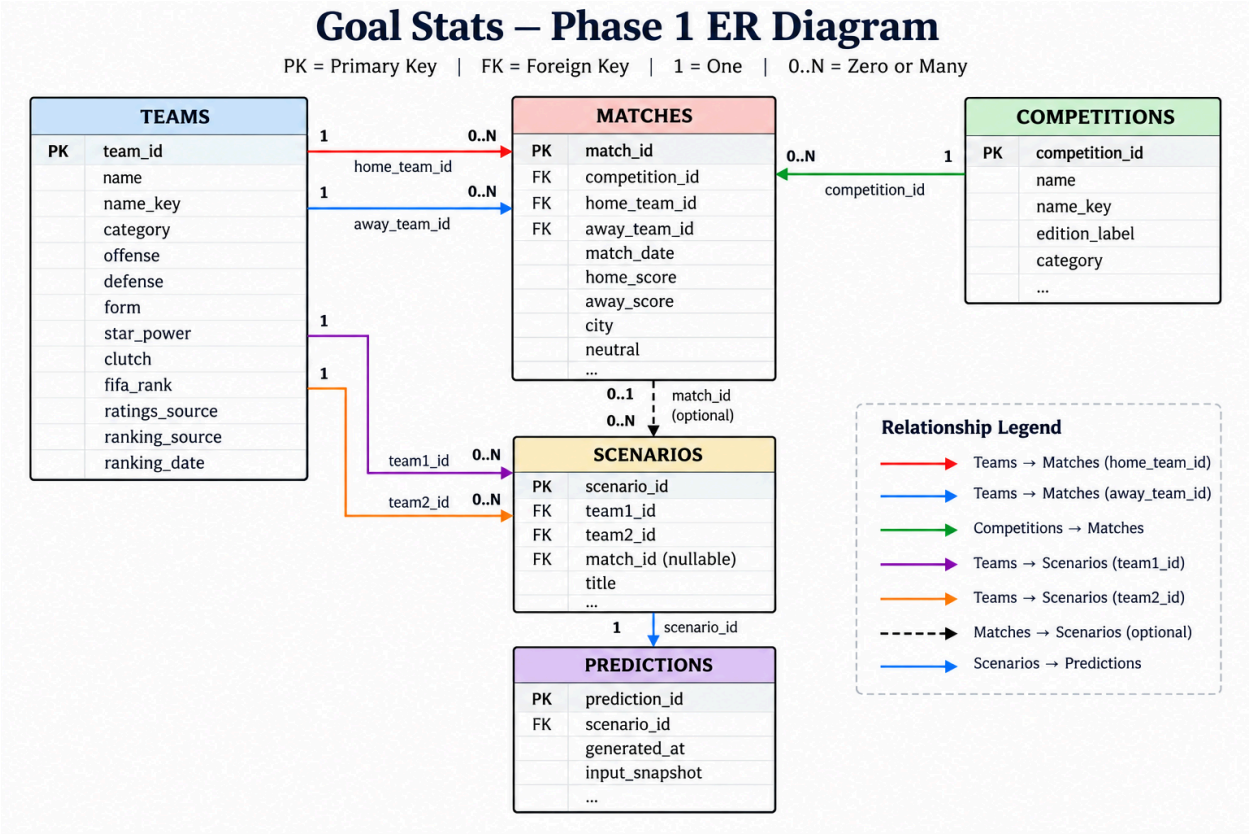
team2_shots_on_target	INTEGER	—	Yes	0–1000
team2_big_chances	INTEGER	—	Yes	0–1000
team2_defender_blocks	INTEGER	—	Yes	0–1000
team2_goalkeeper_saves	INTEGER	—	Yes	0–1000
team2_corners	INTEGER	—	Yes	0–1000
team2_yellow_cards	INTEGER	—	Yes	0–1000
team2_red_cards	INTEGER	—	Yes	0–11
team2_possession	DOUBLE PRECISION	—	Yes	0–100; combined possession rule
team2_modifier_total	DOUBLE PRECISION	—	Yes	−3000–3000; DEFAULT 0
team1_offense	DOUBLE PRECISION	—	Yes	0–100
team1_defense	DOUBLE PRECISION	—	Yes	0–100
team1_form	DOUBLE PRECISION	—	Yes	0–100
team1_star_power	DOUBLE PRECISION	—	Yes	0–100
team1_clutch	DOUBLE PRECISION	—	Yes	0–100
team1_fifa_rank	INTEGER	—	No	Greater than 0 when supplied
team2_offense	DOUBLE PRECISION	—	Yes	0–100
team2_defense	DOUBLE PRECISION	—	Yes	0–100
team2_form	DOUBLE PRECISION	—	Yes	0–100
team2_star_power	DOUBLE PRECISION	—	Yes	0–100
team2_clutch	DOUBLE PRECISION	—	Yes	0–100
team2_fifa_rank	INTEGER	—	No	Greater than 0 when supplied

Predictions

Field	PostgreSQL Data Type	Key	Required	Constraints / Default
prediction_id	INTEGER	PK	Yes	GENERATED BY DEFAULT AS IDENTITY
scenario_id	INTEGER	FK	Yes	References scenarios.scenario_id; ON DELETE RESTRICT
generated_at	TIMESTAMP WITH TIME ZONE	—	Yes	DEFAULT now()
model_version	VARCHAR(80)	—	Yes	Nonblank
team1_win_probability	DOUBLE PRECISION	—	Yes	0–1; combined probability rule
draw_probability	DOUBLE PRECISION	—	Yes	0–1; combined probability rule
team2_win_probability	DOUBLE PRECISION	—	Yes	0–1; combined probability rule
team1_expected_remaining_goals	DOUBLE PRECISION	—	No	Nonnegative and finite when supplied
team2_expected_remaining_goals	DOUBLE PRECISION	—	No	Nonnegative and finite when supplied
most_likely_team1_score	SMALLINT	—	No	Greater than or equal to 0 when supplied
most_likely_team2_score	SMALLINT	—	No	Greater than or equal to 0 when supplied
input_snapshot	JSONB	—	Yes	Must be a JSON object; no default
details	JSONB	—	Yes	Must be a JSON object; no default

ER Diagram & Normalization

The entity relationship (ER) diagram shows how the five PostgreSQL tables (Teams, Competitions, Matches, Scenarios, and Predictions) are connected through primary and foreign keys. The relationships identify required and optional references and show how historical records, hypothetical scenarios, and saved predictions are organized.



Entity	Relationship	Entity
Competitions	1 → 0..N	Matches
Teams	1 → 0..N	Matches (home / away)
Teams	1 → 0..N	Scenarios (team1 / team2)
Matches	1 → 0..N	Scenarios (optional match)
Scenarios	1 → 0..N	Predictions

Normalization

The relational design stores Teams and Competitions separately from historical matches, linking records through primary and foreign keys to reduce unnecessary duplication. Each match references one competition and two teams, while each scenario references two distinct teams and may optionally reference a historical match. Scenarios retain hypothetical rating assumptions and editable match state, while multiple saved predictions can belong to the same scenario. Each prediction preserves its input_snapshot and output details, so earlier results remain interpretable after scenario changes. These deliberate snapshots and historical source fields support traceability and reproducibility rather than eliminating all repeated information.

Database Operations & Application Integration

Database Operations

Goal Stats uses Flask and SQLAlchemy for catalog SELECTs, joined historical match reads, scenario INSERT and PATCH/UPDATE operations, and append-only prediction INSERTs through the API. Prediction previews do not persist results. Historical verification SQL uses GROUP BY, aggregates, and CTEs; runtime prediction endpoints retrieve saved outputs and input snapshots rather than providing a dedicated SQL aggregation API.

Product Function	Database Operation
Browse/select teams	SELECT from Teams
Compare teams	SELECT two Teams and relevant Matches
Browse historical matches	SELECT Matches joined with Teams and Competitions
Create scenario	INSERT into Scenarios
Open saved scenario	SELECT Scenario joined with two Teams; return the optional match_id reference
Generate prediction	Read saved Scenario inputs and Team names; calculate using the Python prediction engine
Save prediction	INSERT into Predictions

View saved prediction	SELECT Prediction, including its stored input_snapshot and output details
Prediction history	SELECT Predictions filtered by scenario_id, ordered by generated_at and prediction_id descending
Basic analytics	Retrieve saved prediction outputs and input snapshots; historical comparisons retrieve match lists

Application Integration

Browser -> Next.js server-side proxy (Vercel) -> Flask REST API (Render) -> SQLAlchemy/psycopg -> PostgreSQL. Flask also invokes the adapted RoadToTheFinal prediction engine.

The frontend calls same-origin /api/football/* routes. The Next.js server-side proxy forwards to Flask /api/v1/* using a server-held token for protected writes. A preview computes without saving. Saving first creates a scenario, then separately generates and inserts a prediction; saved history preserves input snapshots and output even if scenario state is edited later.

Phase 1 Implementation

Goal Stats implements a database-driven application for browsing historical women's football results, comparing teams, creating hypothetical match scenarios, and generating predictions. Users can preview predictions without saving them, save scenarios and predictions, and retrieve previously generated results through the application. The React/Next.js frontend is deployed on Vercel, while the Flask REST API operates on Render using PostgreSQL.

The Phase 1 implementation focuses on these core features. Player-level statistics, lineups, live data feeds, historical backtesting, measured prediction accuracy, and advanced tournament simulation through the frontend are outside the current scope. A separate non-persistent tournament simulation operation exists in the backend.

Database Implementation

The application uses PostgreSQL with five relational tables: Teams, Competitions, Matches, Scenarios, and Predictions. Primary and foreign keys establish relationships, while uniqueness constraints, CHECK constraints, and indexes support data integrity and retrieval.

Historical match records are imported through validated tooling. The import process verifies dataset integrity, checks match records, and preserves source information. Scenario creation and updates use database transactions, including row locking during updates to prevent conflicting changes. Saved prediction generation captures scenario inputs and stores the calculated results in a separate transaction.

Implemented Features

Feature	Phase 1 Implementation
Team Browsing	Retrieve teams and compare available historical results.
Historical Matches	Browse and filter historical Women's World Cup matches.
Scenario Management	Create, retrieve, and update hypothetical match scenarios.
Prediction Preview	Generate prediction results without saving database records.
Saved Predictions	Store generated predictions and their input snapshots.
Prediction History	Retrieve saved predictions associated with scenarios.
Analytics	Display saved prediction outputs and input details.

Testing & Verification

The application was evaluated through automated backend and frontend testing. The following results were recorded on October 17, 2026.

Component	Verification Result
Backend Unit Tests	839 passed
Backend Integration Tests	359 passed
Backend Combined Coverage	98.33%
Frontend Unit Tests	215 passed
Frontend Playwright Tests	9 passed
Frontend Quality Checks	Lint, TypeScript, and production build passed

These results reflect the recorded test execution and do not represent a new live production verification.

User Guide

Goal Stats allows users to explore historical FIFA Women's World Cup results, compare teams, create hypothetical match scenarios, and generate and save predictions. The following instructions describe the Phase 1 application workflow.

1. **Explore Teams:** Select Teams from the navigation menu. Choose two different teams using Team 1 and Team 2 to compare their available ratings, head-to-head results, and historical performances.
2. **Browse Historical Matches:** Select Matches and locate Historical Matches. Filter results using Competition edition and Historical team. Use the previous page and next page to navigate through historical results.
3. **Create a Match Scenario:** Under Select Teams, choose two different teams. Enter five hypothetical strengths for each team: offense, defense, form, star power, and clutch. Each value must be between 0 and 100.
4. **Enter Match Statistics:** Specify Minute, team scores, possession percentages, shots, shots on target, big chances, blocks, saves, corners, and cards. Minutes must be between 0 and 120, possession must total 100%, and shots on target cannot exceed total shots.
5. **Generate a Prediction Preview:** Select Generate Preview to view the predicted win/draw probabilities, likely score, expected remaining goals, and prediction reasoning. The preview is temporary and does not save records.
6. **Save a Prediction:** Select Save new scenario and prediction to store the scenario and its generated prediction. Additional predictions can be saved for an existing scenario without replacing earlier results.
7. **Reopen and Update Scenarios:** On Predictions, locate Saved Scenarios and select Reopen scenario #…. Modify the minute, scores, or match statistics, then select Save scenario changes. Team selections and strength assumptions require a new scenario. Saving changes does not automatically generate another prediction.
8. **Review Prediction History:** Under Saved Scenarios, select Open predictions for scenario #… and open a saved prediction. Use the history controls to browse earlier predictions or reload saved results.
9. **View Analytics:** From a saved prediction, select View Analytics to inspect match momentum, probability history, possession history, expected goals remaining, saved match states, and match statistics. Historical Analytics reflects saved hypothetical predictions rather than live match tracking.

Phase 1 Limitations: Historical records and available team ratings may be incomplete. Prediction previews are not saved automatically. Tournament execution and historical backtesting are deferred. Saved scenarios and predictions are shared within the classroom prototype.

Application Screenshots

The following screenshots show the implemented Goal Stats Phase 1 application. Captured from the deployed website on October 18, 2026; saved match scenarios and predictions are hypothetical.

Figure 1. Goal Stats homepage and application navigation.

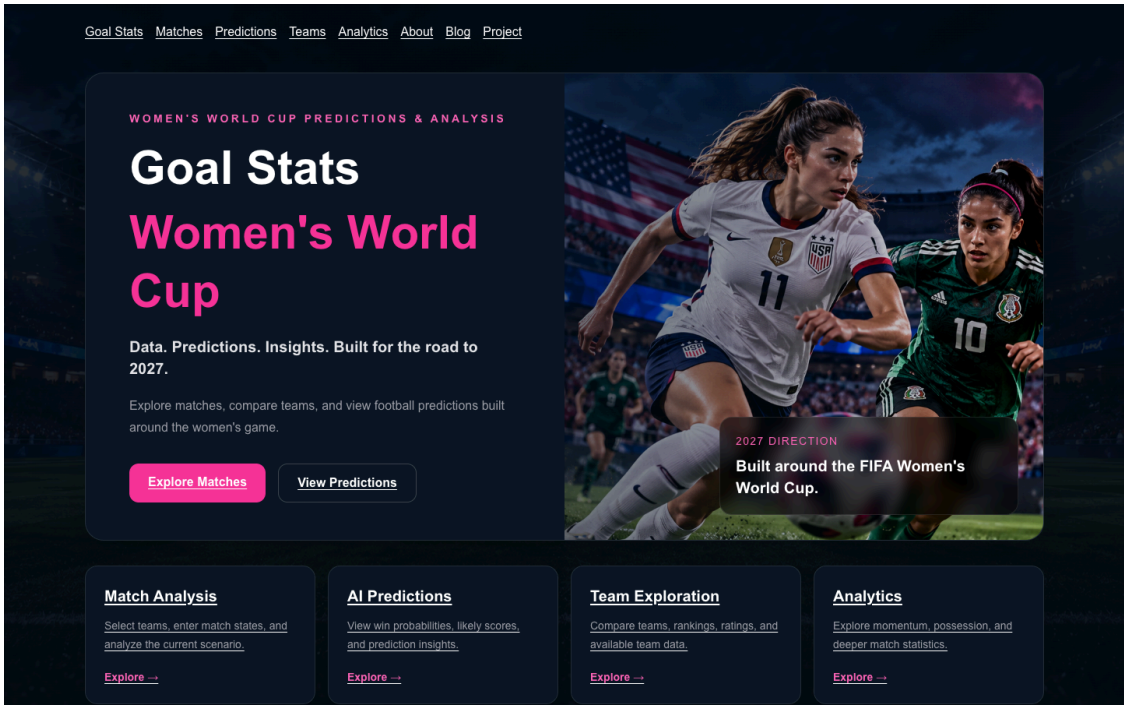


Figure 2. Team comparison for the United States and Japan; unavailable ratings are shown as such.

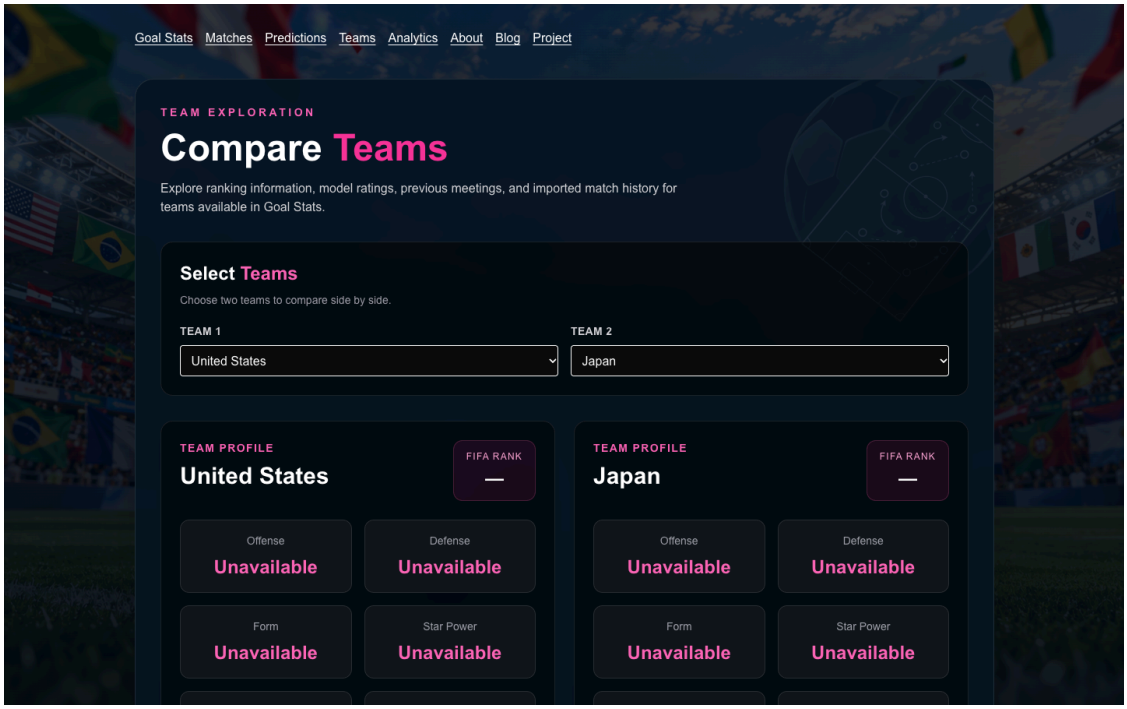


Figure 3. Historical head-to-head results for the United States and Japan.

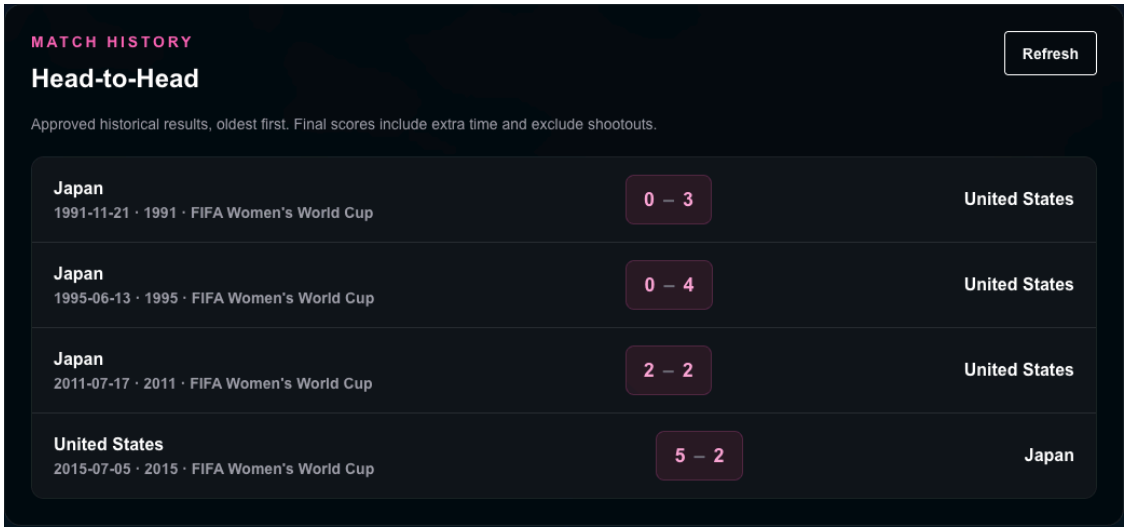


Figure 4. Argentina's 2023 Women's World Cup matches with edition and team filters.

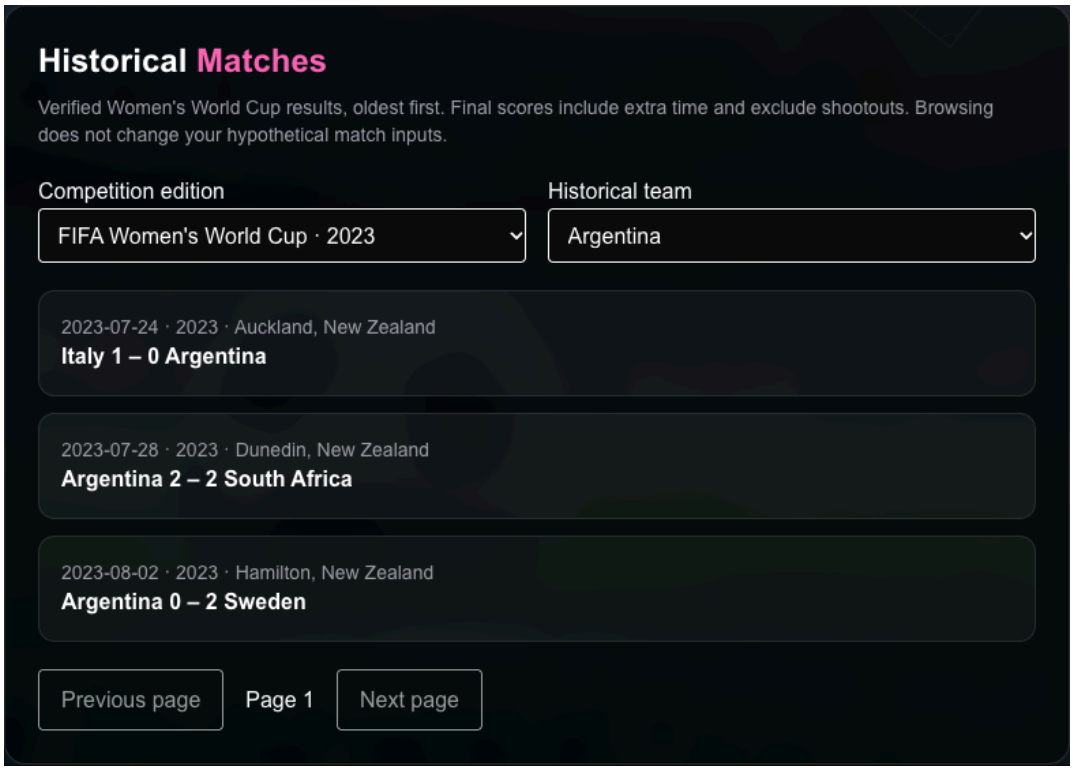
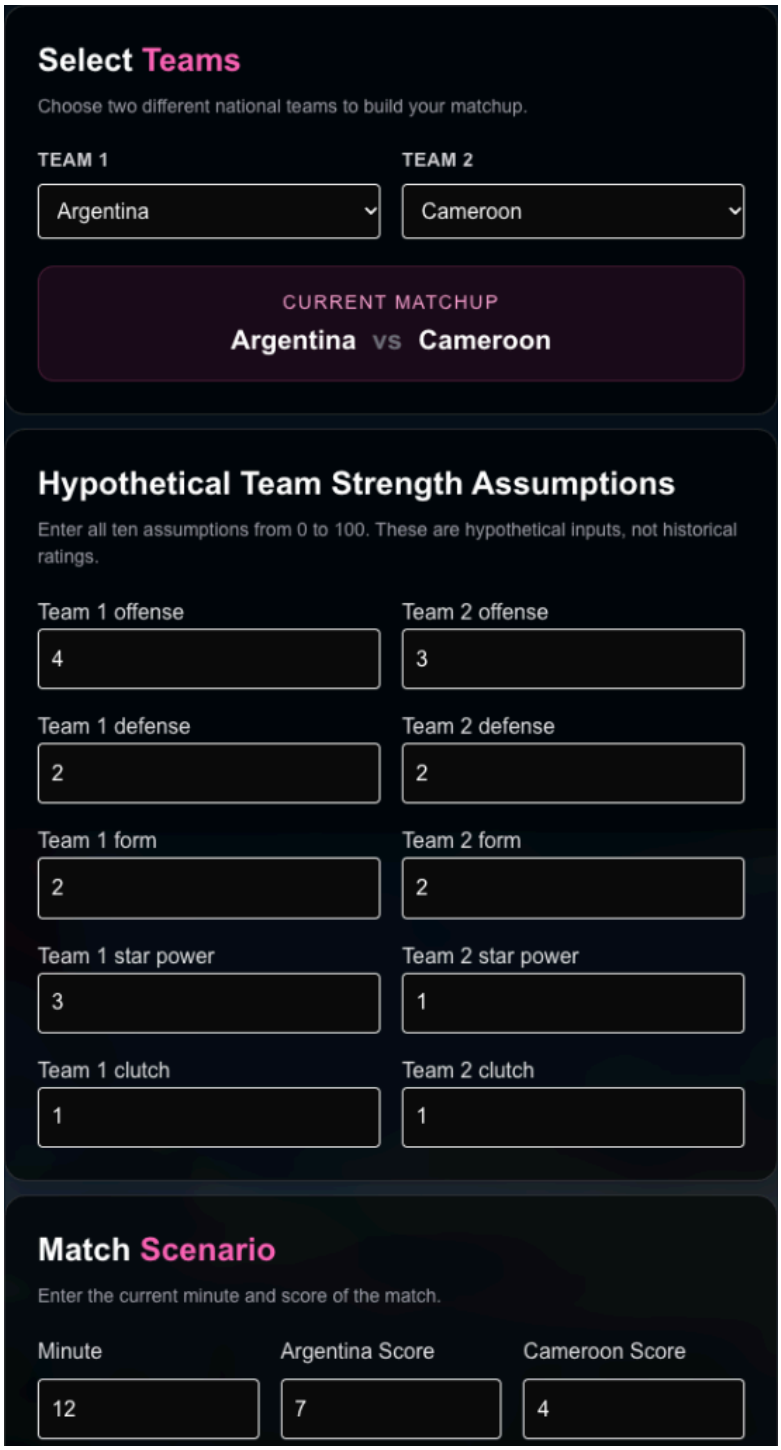


Figure 5. Hypothetical Argentina–Cameroon scenario inputs, team strengths, and match statistics.



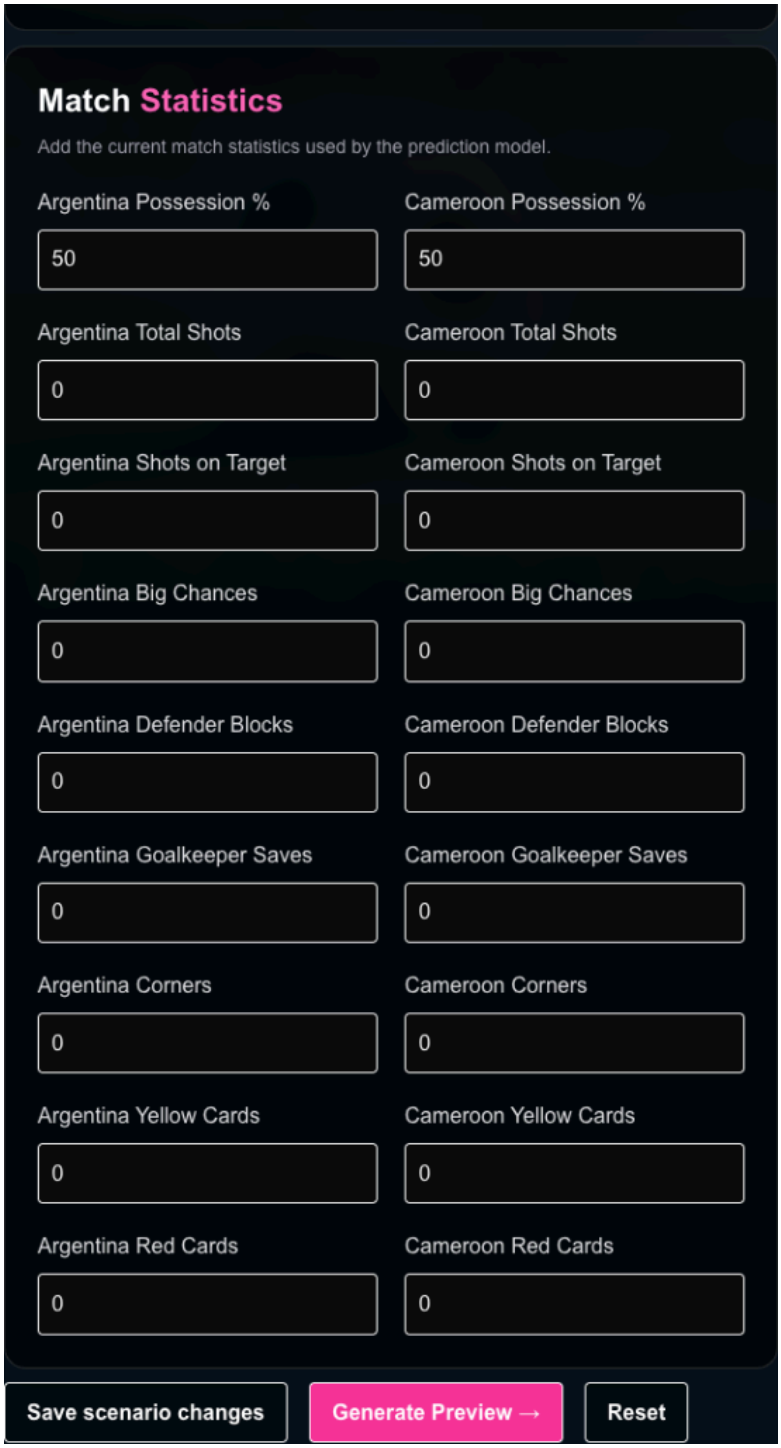

Match Statistics
Add the current match statistics used by the prediction model.
Argentina Possession %
50
Cameroon Possession %
50
Argentina Total Shots
0
Cameroon Total Shots
0
Argentina Shots on Target
0
Cameroon Shots on Target
0
Argentina Big Chances
0
Cameroon Big Chances
0
Argentina Defender Blocks
0
Cameroon Defender Blocks
0
Argentina Goalkeeper Saves
0
Cameroon Goalkeeper Saves
0
Argentina Corners
0
Cameroon Corners
0
Argentina Yellow Cards
0
Cameroon Yellow Cards
0
Argentina Red Cards
0
Cameroon Red Cards
0
Save scenario changes
Generate Preview →
Reset

Figure 6. Saved hypothetical prediction showing outcome probabilities, likely score, and expected goals.



Figure 7. Saved scenario and available prediction history; no prediction is loaded in the displayed panel.

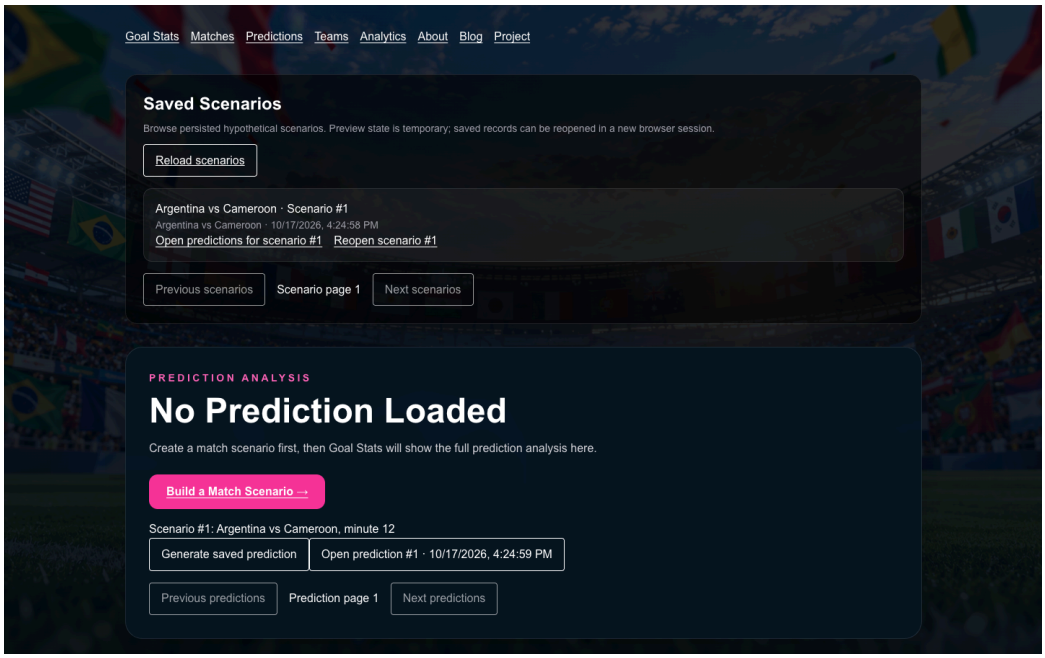
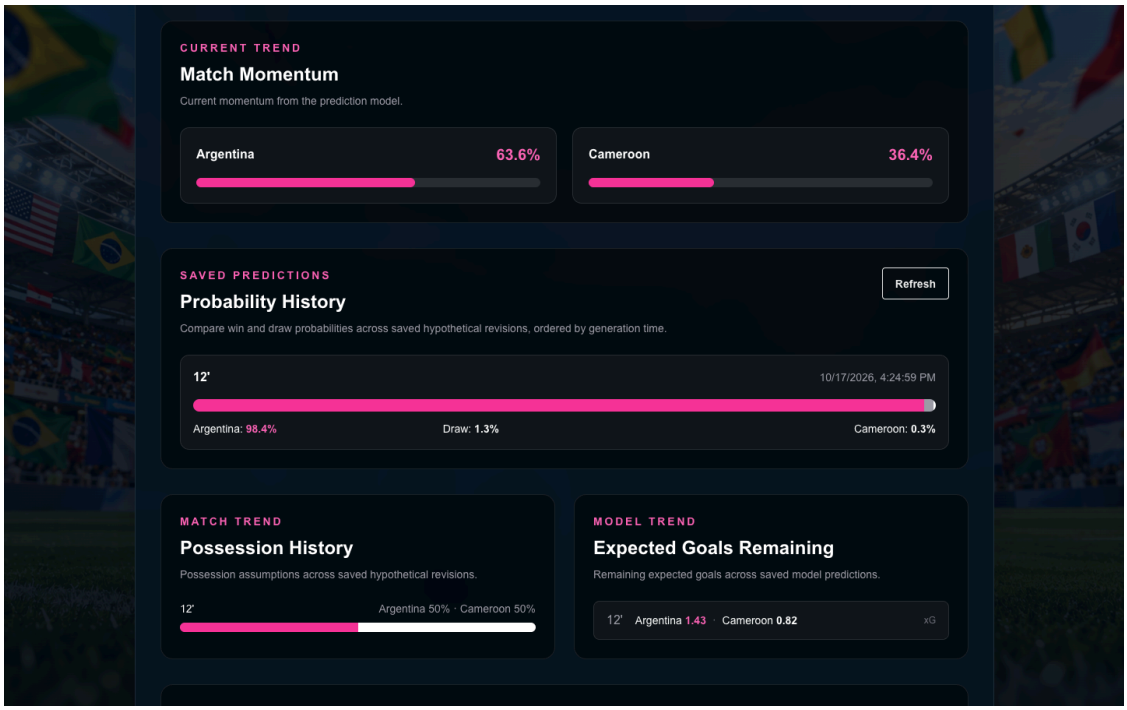


Figure 8. Analytics for one saved hypothetical prediction, including momentum and stored history.



Original Development & AI Assistance

Goal Stats builds on the earlier RoadToTheFinal football prediction prototype. Team Squared adapted its prediction approach and expanded the project into a database-driven application with a React/Next.js frontend, Flask REST API, and PostgreSQL persistence. The Phase 1 implementation adds historical Women's World Cup data import and browsing, relational schema design with constraints and foreign keys, editable hypothetical scenarios, saved prediction records and input snapshots, prediction history, analytics views, automated tests, and cloud deployment. These additions distinguish the submitted database product from the original prototype.

AI Assistance and Team Verification

ChatGPT and Codex were used to assist with brainstorming, development, debugging, refactoring, testing, technical audits, and documentation drafting and formatting. This includes assistance with reviewing schema descriptions, application workflows, and the ER diagram. AI generated suggestions and code were not treated as independent evidence of correctness. The team reviewed and adapted them against the application's source code, SQLAlchemy models, migrations, PostgreSQL schema, automated tests, and observed interface behavior. The team remains responsible for understanding, explaining, and defending the submitted implementation and for correcting any AI generated errors.

References

- **Original prototype**: https://github.com/Estebansito1/RoadToTheFinal
- **Goal Stats frontend**: https://github.com/WestiferRobin/goal-stats-app
- **Goal Stats backend**: https://github.com/WestiferRobin/goal-stats-service
- **Deployed application**: https://goal-stats-app.vercel.app